

Edge Detection Mechanism in CNN

```
# -----
# 6x6 Grayscale Image
# Top Half = Black (0)
# Bottom Half = White (255)
# Apply 3x3 Kernel for Edge Detection
# Show Feature Map
# -----

import numpy as np

# -----
# Step 1 : Create 6x6 Image
# -----

image = np.array([
    [0, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0, 0],
    [255, 255, 255, 255, 255, 255],
    [255, 255, 255, 255, 255, 255],
    [255, 255, 255, 255, 255, 255]
])

print("\nOriginal 6x6 Image")
print(image)

# -----
# Step 2 : 3x3 Kernel for Horizontal Edge Detection
# -----

kernel = np.array([
    [-1, -1, -1],
    [ 0, 0, 0],
    [ 1, 1, 1]
])

print("\n3x3 Kernel")
print(kernel)

# -----
# Step 3 : Convolution Operation
# Output Size = (6-3+1) x (6-3+1) = 4x4
# -----

feature_map = np.zeros((4,4))

for i in range(4):
    for j in range(4):

        # Extract 3x3 region
        region = image[i:i+3, j:j+3]

        # Multiply and Sum
        result = np.sum(region * kernel)

        # Store result
        feature_map[i][j] = result
```

```
# -----
# Step 4 : Show Feature Map
# -----
print("\nFeature Map (Detected Edge)")
print(feature_map)
```

This program demonstrates how a **CNN detects edges** using:

- Input Image
- Kernel / Filter
- Convolution
- Feature Map

Edge Detection

An **edge** means a sudden change in pixel intensity.

Example:

- Black to White
- White to Black
- Dark to Bright

That boundary is called an **edge**.

In this example:

Top 3 rows = Black (0)

Bottom 3 rows = White (255)

So between row 3 and row 4 there is a strong brightness change.

That is the edge.

Step 1 : Input Image (6×6)

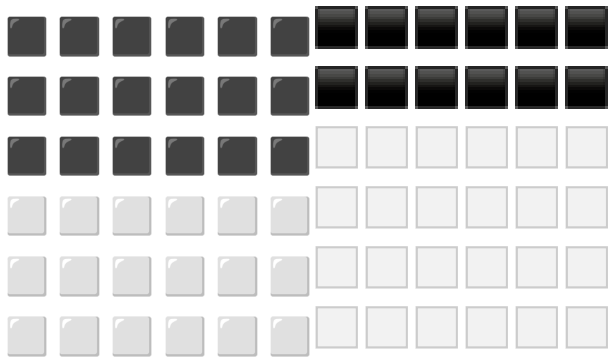
```
image = np.array([
    [0,0,0,0,0,0],
    [0,0,0,0,0,0],
    [0,0,0,0,0,0],
    [255,255,255,255,255,255],
    [255,255,255,255,255,255],
    [255,255,255,255,255,255]
])
```

Meaning

0 = Black pixel

255 = White pixel

So image looks like:



Edge exists in middle.

Step 2 : Kernel / Filter (3×3)

```
kernel = np.array([
    [-1, -1, -1],
    [ 0,  0,  0],
    [ 1,  1,  1]
])
```

Purpose

This kernel detects **horizontal edges**.

It compares:

- Top part
- Bottom part

If top and bottom are different, output becomes high.

How Kernel Works

Top row = negative weights

Middle row = ignored

Bottom row = positive weights

So:

- If top = black and bottom = white
strong positive output
- If same values everywhere
output near zero

Step 3 : Sliding Window (Convolution)

Kernel moves over image one step at a time.

Since:

Image size = 6×6
Kernel size = 3×3

Output size:

$$(6 - 3 + 1) \times (6 - 3 + 1) \\ = 4 \times 4$$

So feature map = 4×4

First Position Calculation

Kernel placed at top-left:

Region:

```
[0 0 0]
[0 0 0]
[0 0 0]
```

Multiply:

$$(0 \times -1) + (0 \times -1) + (0 \times -1) \\ (0 \times 0) + (0 \times 0) + (0 \times 0) \\ (0 \times 1) + (0 \times 1) + (0 \times 1)$$

Total:

0

No edge found.

Important Position (Where Edge Exists)

Kernel on rows 2–4:

Region:

```
[0 0 0]
[0 0 0]
[255 255 255]
```

Multiply:

$$\text{Top row} = 0 \times (-1) + 0 \times (-1) + 0 \times (-1) = 0 \\ \text{Middle row} = 0 \\ \text{Bottom row} = 255 + 255 + 255 = 765$$

Total:

765

Huge value = strong edge detected.

Reason behind Strong Value :

Because sudden change:

0 $\rightarrow$ 255

This brightness jump means boundary line.

Step 4 : Feature Map Output

Likely output:

```
[[ 0 0 0 0]
 [765 765 765 765]
 [765 765 765 765]
 [ 0 0 0 0]]
```

(Depending on positions)

Meaning of Feature Map

- High numbers = edge found
- Low numbers = no edge

So feature map highlights boundaries.

CNN Real Meaning

CNN does same thing on real images:

- Eyes
- Nose
- Shapes
- Object boundaries
- Textures

First layers detect edges.

Later layers detect full objects.

Negative and Positive Values :

Kernel uses contrast checking:

Top darker than bottom → positive
Top brighter than bottom → negative

Magnitude = edge strength.

Summary

Image

Numbers

Kernel

Small detector

Convolution

Move detector over image

Feature Map

Where detector found pattern